

Sisteme de operare

Laborator 5

Semestrul I 2025-2026

Laborator 5

- controlul proceselor
- crearea proceselor
- executia programelor
- terminarea proceselor
- race conditions

Executia programelor

- *fork* creeaza un proces nou, dar codul noului proces e o copie a codului parinte
- apelul sistem *exec* incarca de pe disc imaginea unui program nou care se va suprapune peste imaginea din memorie a procesului apelant
 - Obs: suprapunerea inseamna practic distrugerea procesului apelant
ex: *\$ exec firefox*

- schema tipica de utilizare *exec*:

```

if(!fork()) {
    execl("/bin/ls", "ls", "-l", (char*)0);
    exit(1); // aici nu se ajunge niciodata, decat daca apelul exec a
            // esuat, motiv pt care apelam exit cu cod de eroare
}

// in mod uzual, aici procesul parinte continua singur, daca apelul exec din
// procesul copil a reusit

```

Executia unui program

```
#include <unistd.h>

int execl(const char *pathname, const char *arg0, ... /* (char *) 0 */ );

int execv(const char *pathname, char *const argv[]);

int execl(const char *pathname, const char *arg0, ...
          /* (char *) 0, char *const envp[] */ );

int execve(const char *pathname, char *const argv[], char *const envp[]);

int execlp(const char *filename, const char *arg0, ... /* (char *) 0 */ );

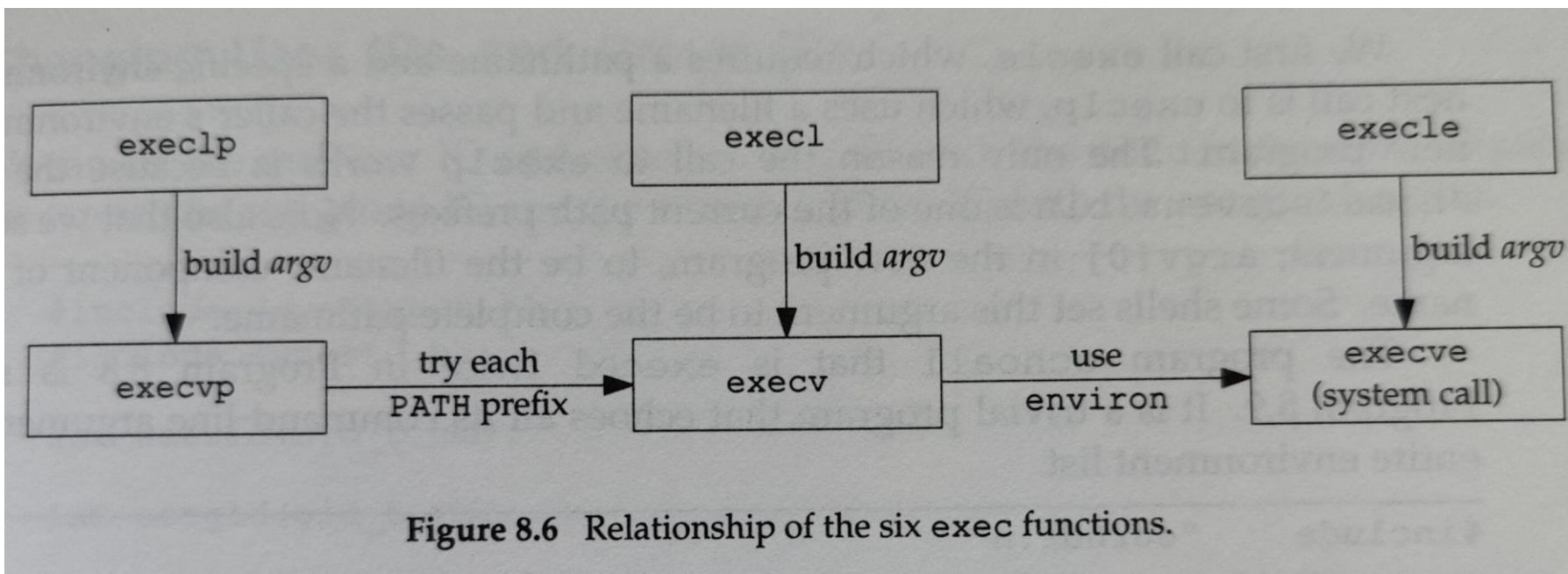
int execvp(const char *filename, char *const argv[]);
```

All six return: -1 on error, no return on success

Diferente apeluri exec

- 6 rutine, doar una singura este apel sistem (*execve*)
- primele 4 folosesc calea absoluta a executabilului, ultimele doua recurg la variabila de mediu PATH pt a identifica executabilul
- exista doua feluri de a specifica parametrul noului program:
 - *lista de argumente*: *execl*, *execlp* si *execle* - lista parametrilor se incheie cu (char *0) sau NULL pointer
 - *vector de argumente*: *execv*, *execvp*, *execve* necesita gruparea argumentelor intr-un vector
- mediul de executie poate fi furnizat:
 - *explicit*: *execve*, *execle* primesc ca parametru de apel un pointer catre lista de variabile de mediu
 - *implicit*: celelalte 4 apeluri al caror nume nu se termina cu "e" acceseaza variabilele de mediu prin intermediul variabilei globale *environ*
*extern char** environ;*

Diferente apeluri exec (cont.)



Proprietati mostenite de la procesul apelant

- dupa *exec*, se pastreaza urmatoarele informatii
 - PID si PPID
 - real UID/GID
 - GID-uri suplimentare
 - GID de proces
 - ID-ul de sesiune,
 - teminalul de control
 - CWD si root directory
 - timpul ramas din alarme
 - umask
 - masca de semnale + semnalele in asteptare
 - limitele referitoare la resurse (v. comanda shell *ulimit*)
 - diversi timpi de executie ai procesului
 - etc.

Functia system

- uneori e util sa executam o comanda shell direct din program

ex: `system("ls -l > out");`

`int system(const char* cmd);`

- *system* este implementata folosind *fork+exec+waitpid*

Script-uri shell

- fișiere text care conțin comenzi interpretate de un interpretor (eventual shell)
- uzual încep cu o linie de forma

`#! <cale interpretor comenzi> [arg optionale]`

ex: `#!/bin/bash`, `#!/usr/bin/python`

- kernelul apelează interpretorul de comenzi cu *fork-exec*
- interpretorul interpretează comenzile din fișierul text
- ex script *myscript.sh*

`#!/bin/bash`

`ls -l > out`

`cat out`

`exit 0`

Script-uri shell (cont.)

- in functie de interpretor, script-urile pot fi programe in sine, cu instructiuni (comenzi de fapt) de tip if-then-else, for, switch, etc
- ex: programul Unix/Linux *awk* folosit de scriptul *myawk.sh* ca interpretor

```
#!/usr/bin/awk -f
```

```
BEGIN {
```

```
    for(i = 0; i < ARGV; i++)
```

```
        printf "ARGV[%d] = %s\n", i, ARGV[i]
```

```
    exit
```

```
}
```

Executia script-urilor

- prin apelul direct al interpretorului potrivit

ex: *\$ sh myscript.sh*

- prin setarea permisiunii de executie pt fisierul script

ex: *\$ chmod u+x myscript.sh*

\$./myscript.sh